

PSEUDOCODE

JAVA PROGRAMMING

NAME:PRETHIVIKA G

REG NO:192521085

DEPT: B.Tech Information Technology

QUESTION 1:

Q1 Student Management System [BL3 – Apply]

10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

Your Answer

```
START
CLASS Student
PRIVATE
    name
    rollno
    marks
    avg
PUBLIC
METHOD input()
    INPUT name
    INPUT rollno
    INPUT n
    FOR(i=0 to n)
        INPUT marks[i]
    END FOR
END METHOD
METHOD calavg()
    sum=0
    FOR(i=0 to n)
        sum=sum+marks[i]
    END FOR
    avg=sum/n
END METHOD
METHOD display()
    PRINT "Student name",name
    PRINT "Roll number",rollno
    PRINT "Average",avg
    IF avg >= 50
        PRINT "Result : Pass"
    ELSE
        PRINT "Result :Fail"
    END IF
END METHOD
END CLASS
BEGIN MAIN
    CREATE std AS Student
    CALL std.input()
    CALL std.calavg()
    CALL std.display()
END MAIN
END
```

QUESTION 2:

Q2 Library Book Management [BL3 – Apply]

10 marks

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.

OOP Concepts: Encapsulation | Getters & Setters | Objects

Your Answer

```
START
CLASS book
PRIVATE
    title
    author
    ISBN
    isavail
PUBLIC
    METHOD setdet(t,a,i)
        title=t
        author=a
        ISBN=i
        isavail=TRUE
    END METHOD
```

```
    METHOD getISBN()
        RETURN ISBN
    END METHOD
    METHOD getavail()
        RETURN isavail
    END METHOD
    METHOD issuebook()
        IF isavail=TRUE THEN
            isavail=FALSE
            PRINT "book issued successfully"
        ELSE
            PRINT "book is already issued"
        END IF
    END METHOD
    METHOD returnbook()
        IF isavail=FALSE THEN
            isavail=FALSE
            PRINT "book returned successfully"
        ELSE
            PRINT "book is already available"
        END IF
    END METHOD
    METHOD displaybook()
        DISPLAY "TITLE: ",gettitle()
        DISPLAY "AUTHOR: ",getauthor()
        DISPLAY "ISBN: ",getISBN()
```

```
        isavail=FALSE
        PRINT "book issued successfully"
    ELSE
        PRINT "book is already issued"
    END IF
END METHOD
METHOD returnbook()
    IF isavail=FALSE THEN
        isavail=FALSE
        PRINT "book returned successfully"
    ELSE
        PRINT" book is already available"
    END IF
END METHOD
METHOD displaybook()
    DISPLAY "TITLE: ",gettitle()
    DISPLAY "AUTHOR: ",getauthor()
    DISPLAY "ISBN: ",getISBN()
    DISPLAY "availability: ",getavail()
END METHOD
END CLASS
```

QUESTION 3:

Q3 Employee Salary Calculation [BL3 – Apply]

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empId, name, basicPay, designation). Implement methods to:

- Calculate gross salary (basic + HRA + DA)
- Calculate net salary after deductions (PF, tax)
- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

Your Answer

```
START
CLASS employee
PRIVATE
    empId
    name
    basicpay
    desig
    hra
    da
    pf
    tax
    grosssal
    net sal
PUBLIC
```

```
PUBLIC
METHOD setemp(id,n,pay,des)
    empId=id
    name=n
    basicpay=pay
    desig=des
END METHOD
METHOD calgross()
    hra=basicpay*20/100
    da=basicpay*10/100
    grossal=basicpay+hra+da
END METHOD
METHOD calnet()
    pf=basicpay*12/100
    tax=grosssal*5/100
    netsal=grosssal-(pf+tax)
END METHOD
METHOD display()
    DISPLAY "emp id: ",empId
    DISPLAY " emp name: ",name
    DISPLAY "designation: ",desig
    DISPLAY "basic pay:",basicpay
    DISPLAY "gross salary: ",grosssal
    DISPLAY "net salary: ",netsal
END METHOD
END CLASS
```

QUESTION 4:

Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]

10 marks

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.

OOP Concepts: Inheritance | Polymorphism | Method Overriding

Your Answer

```
START
CLASS shape
PUBLIC
METHOD calarea()
    DISPLAY "area cannot be calculated"
END METHOD
END CLASS
CLASS circle EXTENDS shape
PRIVATE
    rad
PUBLIC
METHOD setrad(r)
    rad=r
```

```
END METHOD
METHOD calarea()
    area=3.15*rad*rad
    DISPLAY "area of circle: ",area
END METHOD
END CLASS
CLASS rectangle EXTENDS shape
PRIVATE
    len
    bre
PUBLIC
METHOD setdim(l,b)
    len=l
    bre=b
end method
METHOD calarea()
    area=len*bre
    DISPLAY "area of rectangle: ",area
END METHOD
END CLASS
CLASS triangle EXTENDS shape
PRIVATE
    ba
    he
PUBLIC
```

```
PUBLIC
METHOD setdim(b,h)
  ba=b
  he=h
END METHOD
METHOD calarea()
  area=(ba*he)/2
  DISPLAY "area of triangle: ",area
END METHOD
END CLASS
BEGIN MAIN
CREATE c as circle
CREATE r as rectangle
CREATE t as triangle
CALL c.setrad(7)
CALL r.setdim(10,5)
CALL t.setdim(8,6)
CALL c.calarea()
CALL r.calarea()
CALL t.calarea()
END MAIN
END
```



QUESTION 5:

Q5 5. Vehicle Registration System [BL3 – Apply]

10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

OOP Concepts: Inheritance | Method Overriding | Subclass

Your Answer

```
BEGIN
CLASS vehicle
PROTECTED
    reg
    own
    fuel
PUBLIC
METHOD setvehicle(r,o,f)
    reg=r
    own=o
    fuel=f
END METHOD
METHOD display()
    DISPLAY "registration no: ",reg
    DISPLAY "owner: ",owner
    DISPLAY "owner: ",owner
    DISPLAY "fuel type: ",fuel
END METHOD
END CLASS
CLASS twowheel EXTENDS vehicle
PRIVATE
    engcap
PUBLIC
METHOD seteng(cap)
    engcap=cap
END METHOD
METHOD display()
    DISPLAY "registration no: ",reg
    DISPLAY "owner: ",owner
    DISPLAY "fuel type: ",fuel
    DISPLAY "engine capacity: ",engcap
END METHOD
END CLASS
CLASS fourwheel EXTENDS vehicle
PRIVATE
    seat
    ac
PUBLIC
METHOD setfour(se,acc)
    seat=se
    ac=acc
```



```
seat=se
ac=acc
END METHOD
METHOD display()
    DISPLAY "registration no: ",reg
    DISPLAY "owner: ",owner
    DISPLAY "fuel:",fuel
    DISPLAY "seating capacity ",seat
    DISPLAY "ac status",ac
END METHOD
END CLASS
BEGIN MAIN
CREATE
CREATE
CALL b.setvehicle("TN01AB","Rahul","petrol")
CALL b.seteng(150)
CALL c.setvehicle("TN09CD","OVIYA","diesel")
CALL c.setfour(5,"yes")
CALL b.display()
CALL c.display()
END MAIN
END
```

QUESTION 6:

Q6 6. Bank Account Hierarchy [BL4 – Analyze]

10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

OOP Concepts: Inheritance | Method Overriding | Polymorphism

Your Answer

```
BEGIN
CLASS bankacc
PROTECTED
    accno,holder,bal
PUBLIC
METHOD setdet(a,h,b)
    accno=a
    holder=h
    bal=b
END METHOD
METHOD Deposit(amt)
    bal=bal+amt
END METHOD
```

```
METHOD withdraw(amt)
    DISPLAY " withdrawal"
END METHOD
METHOD display()
    DISPLAY "acc num",accno
    DISPLAY "Holder",holder
    DISPLAY "balance",bal
END METHOD
END CLASS
CLASS saving EXTENDS bankacc
PRIVATE
    rate
PUBLIC
METHOD setint(r)
    rate=r
END METHOD
METHOD calint()
    inrt=bal+rate/100
    bal=bal+inrt
END METHOD
METHOD withdraw(amt)
    IF amt<=bal THEN
        bal=bal-amt
        DISPLAY "withdrawal successful"
    ELSE
        DISPLAY "insufficient balance"
```

```
END METHOD
END CLASS
CLASS curacc EXTENDS bankacc
PRIVATE
    ovl
PUBLIC
METHOD withdraw(amt)
    IF amt <= (bal+ovl) THEN
        bal=bal-amt
        DISPLAY "withdrawal successful"
    ELSE
        DISPLAY "overdraft limit exceeded"
    END IF
END METHOD
END CLASS
BEGIN MAIN
CREATE s as saving
CREATE c as curacc
CALL s.setdet(1001,"RAHUL",20000)
CALL s.setint(5)
CALL c.setdet(2001,"priya",10000)
CALL c.setint(5000)
CALL s.calint()
CALL s.withdraw(5000)
CALL c.withdraw(12000)
CALL s.display()
```

QUESTION 7:

Q7 7. Hospital Patient Record System [BL4 – Analyze]

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

Your Answer

```
BEGIN
CLASS patient
PROTECTED
    pid.name
PRIVATE
    dig.pres
PUBLIC
METHOD setdet(id,n,d,p)
    pid=id
    name=n
    dig=d
    pres=p
END METHOD
METHOD genbill()
END METHOD
```

```
END CLASS
CLASS inpatient Extends patient
PRIVATE
    ward.admnd.dch.day
PUBLIC
METHOD setinpatient(w,a,c,d)
    ward=w
    admnd=a
    dch=c
    day=d
END METHOD
METHOD genbill()
    bill=dch*day
    DISPLAY "inpatient bill: ",bill
END METHOD
END CLASS
CLASS outpatient EXTENDS patient
PRIVATE
    appda.cfee
PUBLIC
METHOD setoutpatient(a,f)
    appda=d
    cfee=f
END METHOD
METHOD genbill()
    bill=cfee
```

```
METHOD setoutpatient(a,t)
  appda=d
  cfee=f
END METHOD
METHOD genbill()
  bill=cfee
  DISPLAY "outpatient bill: ",bill
END METHOD
END CLASS
BEGIN MAIN
CREATE ip as inpatient
CREATE op as outpatient
CALL ip.setdet(101,"rahul","fever","medicine")
CALL ip.setinpatient(12,"10/07/2026",2000,5)
CALL op.setdet(102,"priya","cold","tablets")
CALL op.setoutpatient("12/07/2026",500)
CALL ip.genbill()
CALL op.genbill()
END MAIN
END
```

QUESTION 8:

Q8 8. E-Commerce Product Catalog [BL4 – Analyze]

10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

Your Answer

```
gen=g
END METHOD
METHOD applydis()
    price=price-(price*20/100)
    DISPLAY "clothing price",price
END METHOD
END CLASS
CLASS grocery EXTENDS product
PRIVATE
    expd,wei
PUBLIC
METHOD setgro(e,w)
    expd=e
    wei=w
END METHOD
```

```
END METHOD
METHOD applydis()
    price=price-(price*5/100)
    DISPLAY "grocery price",price
END METHOD
END CLASS
BEGIN MAIN
CREATE e as elect
CREATE c as clothing
CREATE g as grocery
CALL e.setdet(101,"laptop",60000)
CALL e.setelect(2,"dell",230)
CALL c.setdet(102,"shirt",1500)
CALL c.setcloth("l" "cotton" "men")
CALL g.setdet(103,"rice",1200)
CALL g.setgro("15/08/2026","10")
CALL e.applydis()
CALL c.applydis()
CALL g.applydis()
END MAIN
END
```

QUESTION 9:

Q9 9. University Staff Hierarchy [BL4 – Analyze]

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

Your Answer

```
BEGIN
CLASS staff
PROTECTED
    sid,name,dept
PRIVATE
    sal
PUBLIC
METHOD setdet(id,n,d,s)
    sid=id
    name=n
    dept=d
    sal=s
END METHOD
```

```
METHOD calall()
    DISPLAY "base allowance"
END METHOD
END CLASS
CLASS teach EXTENDS staff
PROTECTED
    sub
PUBLIC
METHOD calall()
    DISPLAY "teaching allowance=5000"
END METHOD
END CLASS
CLASS proff EXTENDS teach
PRIVATE
    resall
PUBLIC
METHOD calall()
    DISPLAY "professor allowance=1000"
END METHOD
END CLASS
CLASS lecturer EXTENDS teach
PRIVATE
    lehr
PUBLIC
METHOD calall()
    DISPLAY "lecturer allowance=7000"
```

```
    DISPLAY "lecturer allowance=7000"
END METHOD
END CLASS
CLASS nonteach EXTENDS staff
PRIVATE
    des
PUBLIC
METHOD calall()
    DISPLAY "non-teaching allowance=4000"
END METHOD
END CLASS
BEGIN CLASS
CREATE p as proff
CREATE l as lecturer
CREATE n as nonteach
CALL p.setdet(101,"rahul","cse",80000)
CALL l.setdet(102,"priya","it",50000)
CALL n.setdet(103,"arun","admin",40000)
CALL p.calall()
CALL l.calall()
CALL n.calall()
END MAIN
END
```


QUESTION 10:

Q10 10. Smart Home Device Controller [BL4 – Analyze]

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

Your Answer

```
BEGIN
CLASS smartdev
PROTECTED
    devid
    devname
PUBLIC
METHOD setdet(id name)
    devid=id
    devname=name
END METHOD
METHOD execom(cmd)
END METHOD
END CLASS
```

```
END CLASS
CLASS smlight EXTENDS smartdev
PUBLIC
METHOD execom(cmd)
    IF cmd="on" THEN
        DISPLAY "light turned on"
    ELSE IF cmd="off" THEN
        DISPLAY "light turned off"
    ELSE
        DISPLAY "light status displayed"
    END IF
END METHOD
END CLASS
CLASS smarttherm EXTENDS smartdev
PUBLIC
METHOD execom(cmd)
    IF cmd="on" THEN
        DISPLAY "thermostat activated"
    ELSE IF cmd="off" THEN
        DISPLAY "thermostat deactivated"
    ELSE
        DISPLAY "temperature status displayed"
    END IF
END METHOD
END CLASS
CLASS smartlock EXTENDS smartdev
```

```

PRIVATE
    pin
PUBLIC
METHOD setpin(p)
    pin=p
END METHOD
METHOD execom(cmd)
    IF cmd="on" THEN
        DISPLAY "door locked"
    ELSE IF cmd="off" THEN
        DISPLAY "door unlocked"
    ELSE
        DISPLAY "lock status dispalved"
    END IF
END METHOD
END CLASS
BEGIN MAIN
CREATE li as smlight
CREATE th as smarttherm
CREATE lo as smartlock
CALL li.setdet(101,"bedroom light")
CALL th.setdet(102,"hall thermostat")
CALL lo.setdet(103,"main door lock")
CALL lo.setpin("1234")
CALL li.execom("on")
CALL th.execom("status")
CALL lo.execom("off")

```

```

        DISPLAY "door unlocked"
    ELSE
        DISPLAY "lock status dispalved"
    END IF
END METHOD
END CLASS
BEGIN MAIN
CREATE li as smlight
CREATE th as smarttherm
CREATE lo as smartlock
CALL li.setdet(101,"bedroom light")
CALL th.setdet(102,"hall thermostat")
CALL lo.setdet(103,"main door lock")
CALL lo.setpin("1234")
CALL li.execom("on")
CALL th.execom("status")
CALL lo.execom("off")
END MAIN
END

```